

MegaScale-MoE：生产环境中面向大规模专家混合模型的通信高效训练

Chao Jin^{*§}, Ziheng Jiang^{*†}, Zhihao Bai[†], Zheng Zhong[†], Juncai Liu[†], Xiang Li[†], Ningxin Zheng[†],
Xi Wang[†], Cong Xie[†], Qi Huang[†], Wen Heng[†], Yiyuan Ma[†], Wenlei Bao[†], Size Zheng[†], Yanghua
Peng[†], Haibin Lin[†], Xuanzhe Liu[§], Xin Jin[§], Xin Liu[†]

[§] 北京大学计算机学院 [†]ByteDance Seed

摘要

我们提出 MegaScale-MoE，这是一个面向生产环境的系统，专门用于高效训练大规模专家混合（Mixture-of-Experts, MoE）模型。MoE 已成为一种很有前景的架构，可将大语言模型（Large Language Models, LLMs）扩展到前所未有的规模，从而提升模型性能。然而，现有 MoE 训练系统会出现训练效率下降的问题；随着 MoE 模型规模持续增大、硬件不断演进，这一问题进一步加剧。

认识到高效通信对提升 MoE 训练的重要作用，MegaScale-MoE 为每个 MoE 层中的注意力和 FFN 定制了通信高效的并行策略，并以整体化方式在算子间和算子内两个层面重叠通信与计算。此外，MegaScale-MoE 结合调整后的通信模式，将通信压缩到更低精度，进一步提升训练效率。在 1,440 张 NVIDIA Hopper GPU 上训练 352B MoE 模型时，MegaScale-MoE 达到 1.41M tokens/s 的训练吞吐，相比 Megatron-LM 将效率提升了 1.88×。我们分享加速 MoE 训练的实际运维经验，并希望通过给出系统设计中的见解，推动未来 MoE 系统研究。

CCS Concepts: • Computer systems organization → Cloud computing; • Computing methodologies → Machine learning; • Networks → Data center networks.

Keywords: 专家混合模型，分布式训练，计算通信重叠

ACM 引用格式:

Chao Jin, Ziheng Jiang, Zhihao Bai, Zheng Zhong, Juncai Liu, Xiang Li, Ningxin Zheng, Xi Wang, Cong Xie, Qi Huang, Wen Heng, Yiyuan Ma, Wenlei Bao, Size Zheng, Yanghua Peng, Haibin Lin, Xuanzhe Liu, Xin Jin, Xin Liu. 2025. MegaScale-MoE: Large-Scale Communication-Efficient Training of Mixture-of-Experts Models

* 同等贡献。

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

EUROSYS '26, April 27–30, 2026, Edinburgh, Scotland UK

© 2026 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 979-8-4007-2212-7/26/04...\$15.00

<https://doi.org/10.1145/3767295.3769325>

in Production. In *EuroSys '26, April 27 – 30, 2026, Edinburgh, UK*. ACM, New York, NY, USA, 17 pages. <https://doi.org/10.1145/3767295.3769325>

1 引言

随着大语言模型（LLMs）[7, 18, 49] 的规模增长，其训练规模也随之扩大。训练规模的上升使得效率提升不再只是值得追求，而是变得至关重要 [19]。作为一家为数亿用户构建 AI 产品的公司，我们持续投入在数千张 GPU 上训练拥有数千亿参数的 LLM。因此，即使训练效率只有边际提升，也能显著减少计算资源消耗和训练时间，并直接影响开发最先进 LLM 的可行性和可持续性。

在 LLM 架构中，专家混合（MoE）模型因稀疏激活而突出 [7, 10, 18, 46]：它不会让输入 token 经过所有参数，而是将其动态路由到一组被选中的专门网络组件，即专家。随着模型规模增大，这种设计使所需 FLOPs 呈次线性增长，从而显著降低计算成本。近期工业界进展 [2, 3, 9, 27, 40] 展示了 MoE 模型的潜力：在达到相当模型质量的情况下，相比稠密模型可将训练成本降低一个数量级。

尽管 MoE 模型训练成本更低，但从系统角度看，我们观察到训练中的关键性能瓶颈是通信。例如，在 NVIDIA Hopper GPU 上训练一个内部模型时，通信占前向传播总时间的 43.6%，并占整个训练过程的 32%。这一瓶颈主要来自两个因素。第一，MoE 模型天然引入更多通信开销。相比稠密模型训练，MoE 模型由于参数规模更大，需要分布到更多 GPU 上进行模型并行。第二，为了支持稀疏计算，前向和反向传播中都需要额外两次 all-to-all 通信，分别用于分发和聚合 token，这会阻碍正在进行的计算。

此外，随着硬件进步，计算与通信之间的不平衡越来越明显，通信开销变得更加主导。伴随模型架构改进，硬件能力也快速演进，GPU 处理速度显著提高（图 1）。与此同时，训练精度降低也被用于提升训练效率和成本效益 [27, 38]。这些趋势共同导致原始计算时间下降，使通信开销的相对影响成为更关键的瓶颈。例如，我们观察到，在某些情况下，仅将现有张量并行扩展到多节点设置就会使通信开销超过 50%。因此，优化通信对于维持并提升 MoE 模型训练的可扩展性至关重要，尤其是在需要跨多张 GPU 频繁进行数据同步的分布式环境中。

本文介绍 MegaScale-MoE 的设计、实现和运维经验。它是一个面向高效大规模 MoE 训练优化的生产系统。通过细致处理通信瓶颈，MegaScale-MoE 试图推进 MoE

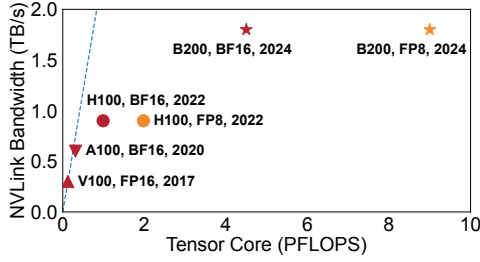


Figure 1. NVIDIA GPU 的演进。

训练的边界，并在性能和效率上取得显著提升。我们的洞察是：MoE 与稠密模型之间最关键的架构差异位于层内，而这正是通信开销的主要来源。基于这一点，MegaScale-MoE 将每个 MoE 层限制在单节点内，利用高带宽 NVLink。我们的分析 (§3) 和评估 (§6) 表明，尽管现有系统 [15, 27] 常采用跨节点专家并行，我们的方法仍能将 MoE 训练有效扩展到数千张 GPU 上的数千亿参数模型。

具体而言，MegaScale-MoE 从三个关键方面处理 MoE 训练中的通信问题。第一，MegaScale-MoE 为每个 MoE 层中的注意力模块和 FFN 模块定制并行策略，以减少通信量。我们比较现有 LLM 训练框架中的并行策略，并全面考虑它们对大规模训练的影响，包括通信量，以及通信能否被有效重叠（即是否位于关键路径上）。基于这一分析，我们为 MoE 训练选择最优的并行策略组合。

第二，MegaScale-MoE 在算子层面充分重叠通信与计算。MegaScale-MoE 将每个 MoE 层的前向和反向传播划分为不同的计算算子与通信算子。对于算子间重叠，MegaScale-MoE 采用整体化调度策略，在前向和反向传播中精心重排通信和计算算子，将通信隐藏在独立计算之中。该方法还优化了 GPU 内存使用。MegaScale-MoE 使用选择性激活重物化，在前向传播中仅将部分激活保留在 GPU 内存中，并在反向传播中通过重算或重新通信获得所需激活。借助这种整体化调度，MegaScale-MoE 有效隐藏重物化开销，在只存储一半激活的同时实现相近性能。

为了重叠关键路径上的通信，MegaScale-MoE 采用细粒度方法，将通信拆成 tile，并与 GPU 计算模式对齐，把这些 tile 级通信融合进计算 kernel。对于带有 token 分发的 MoE 模型，MegaScale-MoE 将高效的本地 scatter 操作融合进 kernel，并沿 scatter 后的维度重组计算任务，以缓解来自多个数据源的通信瓶颈。这种细粒度重叠发生在每个节点内部，利用 GPU 之间的高带宽连接。

第三，MegaScale-MoE 利用通信压缩进一步提升 MoE 训练效率。具体而言，对于广泛使用的 BF16 混合精度训练，MegaScale-MoE 将跨节点参数同步精度从 FP32 降到 BF16，使相关开销减半。在 FP8 训练中，MegaScale-MoE 用 FP8 通信替代 BF16 reduce-scatter，并结合定制化策略和 FP32 归约，在保持收敛稳定性的同时降低通信量。

MegaScale-MoE 已部署在我们的数据中心，用于为产品训练 MoE 模型。相比最先进的开源 LLM 训练框架 Megatron-LM [48]，在 1,440 张 NVIDIA Hopper GPU 上训练 352B MoE 模型时，MegaScale-MoE 的 MFU (Model FLOPs Utilization) 最高提升 1.88×。凭借全面的通信优

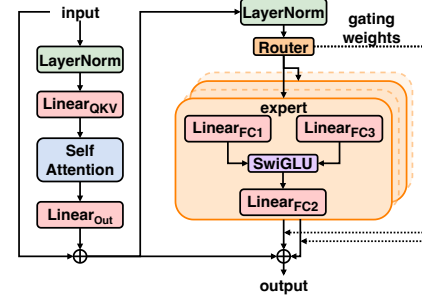


Figure 2. 专家混合 (MoE) 层。

化，MegaScale-MoE 支撑了我们生产环境中的大规模训练，高效扩展到万亿参数和数千张 GPU，并节省了数百万 GPU 小时。

2 背景

2.1 面向 Transformer 的专家混合

专家混合 (MoE) 机制是一种先进方法，旨在提升 Transformer [51] 模型的性能；Transformer 在 LLM 领域中正变得日益关键 [2, 7, 18, 27]。MoE 通过在前馈网络 (FFN) 组件中集成多个专家网络来扩展 Transformer 架构。如图 2 所示，MoE 模型会根据输入 token 的特征，将其动态路由到最相关的专家。该路由由可训练的 gating 机制管理，为每个 token 选择最合适的专家。由于每个输入只激活一部分专家，这种架构创新使 MoE 模型能够扩大容量，而不会使推理成本按比例增长。

2.2 大规模 LLM 训练

在数万张 GPU 上大规模训练大语言模型是一项复杂的系统工程挑战，需要多种系统技术。为了分布训练负载，必须结合数据并行、张量并行和流水线并行等并行策略 [19, 43, 48]，因为每种方法都有局限，单独依赖某种方法无法有效扩展。

数据并行将训练数据均匀分布到所有设备上，每个设备都复制模型参数和优化器状态。为了在每次训练迭代后同步参数，数据并行会执行 all-reduce 通信操作。Zero Redundancy Optimizer (ZeRO) [41] 通过在所有参与设备之间分布模型状态来改进数据并行。ZeRO 分为三个递进阶段，每个阶段都进一步节省内存，但代价是通信开销增加。

张量并行将计算密集型张量操作分布到多个设备上，从而实现并行计算并显著加速训练过程。具体的划分策略以及模型内部算子之间的依赖关系，决定了张量并行可能需要收集被切分的输入 (all-gather) 或合并输出 (reduce-scatter)。在 LLM 训练中，LayerNorm 和 Dropout 等算子虽然计算不密集，却需要大量激活内存。为解决这一问题，研究者提出了称为序列并行的张量并行变体 [20]，沿序列长度维度划分这些算子。对于长上下文训练，一些工作 [1, 16, 48] 将序列并行或张量并行应用到自注意力中的不同算子。图 3 展示了注意力的主流并行策略，即张量并行、序列并行和上下文并行 (TP、SP 和 CP)，我们将在 §3.1 中分析它们。

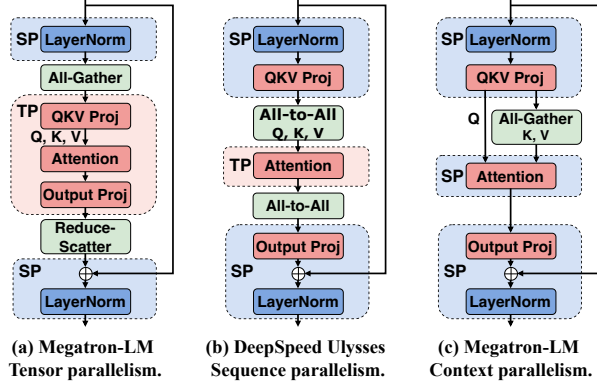


Figure 3. 自注意力的不同并行策略。“TP”表示沿隐藏维度划分，“SP”表示沿序列长度维度划分。

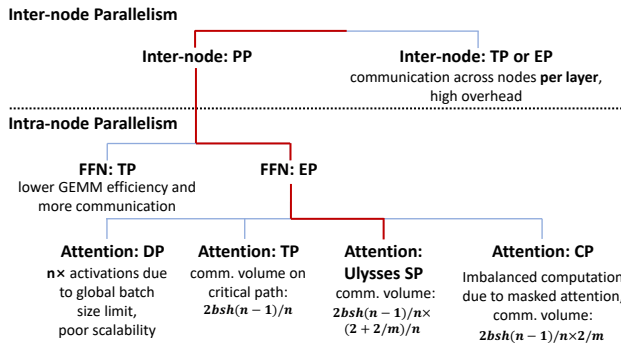


Figure 4. 大规模 MoE 训练的设计空间。

流水线并行通过将模型层划分为不同阶段并在不同设备上处理来提升效率，从而实现流水线执行。为此，每个 batch 会被拆分为多个 micro-batch。为了减少流水线气泡，已有多种调度策略被提出，例如 GPipe [14]、PipeDream 1F1B [33] 和 Interleaved 1F1B [34] 等。Megatron-LM 采用 Interleaved 1F1B 流水线调度，将一个设备上的每个阶段进一步划分为多个虚拟阶段，以降低流水线气泡率。

专家并行专门用于训练 MoE 模型，它将专家分布到多个设备上，从而缓解内存压力并支持并行处理。为了高效地将 token 分配给合适的专家并取回其输出，通常会使用 all-to-all 通信。

3 通信高效的并行策略

随着 MoE 模型兴起以及硬件计算能力演进，通信开销在生产环境 MoE 训练中变得越来越关键。本节深入讨论用于降低通信量并满足其他训练需求的并行策略，例如保持较高的 GEMM (General Matrix Multiplication) 效率。

图 4 展示了大规模 MoE 训练的并行策略设计空间，其中不包括最外层的数据并行。我们先看跨节点并行。专家并行通过将专家分布到多个节点上，缓解 MoE 模型参数规模巨大带来的内存压力，但也会引入逐层跨节点通信，从而损害训练效率。类似地，张量并行的高通信开销使得将 TP 限制在单节点内更有效。沿用先前工

符号	描述
b	micro-batch 大小
s	序列长度
h	隐藏维度大小
n	模型并行 (TP、SP 或 EP) 规模
m	query heads 数量与 key-value heads 数量之比
k	每个 token 被路由到的专家数量

Table 1. 符号说明。

作 [19] 的做法，我们采用流水线并行来分布模型参数、减少通信，并重叠不同 micro-batch 的通信。

以 Megatron-LM [48] 和 DeepSpeed-MoE [40] 为代表的既有大规模 MoE 训练系统，通过在节点内切分模型参数并引入张量并行来扩展训练。然而，在实践中我们观察到这种方法存在两个问题：(1) TP 会切分专家维度，负面影响 GEMM 效率；(2) TP 会引入显著通信开销，并且该开销不会随着并行规模增大而下降，最终在现代硬件上使通信超过计算。

为解决这些问题，我们针对 MoE 模型组件定制并行策略。对于前馈网络（即专家），我们用专家并行替代张量并行，并针对不同 top-k 和专家规模使用定制通信模式，确保通信开销低于张量并行。对于其他组件，我们采用序列并行，沿序列维度而不是 batch 维度进行划分，从而在不增大全局 batch size 的情况下扩展训练。与张量并行相比，这也降低了关键路径上的通信。由于不同组件之间存在参数不对称，额外内存和 DP 通信开销仍然可控。后续小节将详细说明这一节点内并行策略的理由和分析。表 1 列出了关键符号。

3.1 用于注意力的序列并行

由于 MoE 模型中的专家组件天然易于并行化，大多数 MoE 训练相关工作 [22, 40] 都聚焦于优化专家并行，而对注意力等非 MoE 组件通常采用数据并行 (DP)。然而，在扩展 MoE 训练时，由于 $n \times$ 的激活内存消耗，这种方法并不充分。问题源于 DP 在节点间和节点内都切分 batch 维度。相比图 4 中的其他节点内并行策略，将 DP 用于注意力会迫使节点内每张 GPU 同时处理一个 micro-batch，使激活大小增加 $8 \times$ ，常常导致显存不足。

为了实现可扩展的 MoE 训练，在注意力模块上实现节点内并行至关重要。张量并行 (TP) 常被用于在节点内并行化注意力操作。然而，它会因在关键路径上对激活执行 all-gather 和 reduce-scatter 而引入不可避免的通信成本。随着计算 FLOPs 与通信带宽之间的差距扩大，我们发现 TP 的通信开销甚至可能超过自注意力的计算时间。这种由通信主导的瓶颈限制了通信与计算重叠的空间，最终降低训练效率。

我们采用 DeepSpeed-Ulysses [16] 提出的序列并行 (SP) 来扩展 MoE 训练，并有效降低关键路径上的通信。SP 常用于长上下文训练，用以解决长输入带来的内存挑战。我们发现它同样适用于大规模 MoE 训练。首先，与 TP 相比，它显著降低通信开销，尤其是在使用 grouped-query attention [4] 时。其次，虽然它会引入一定参数冗余，并在参数同步时增加通信开销，但 MoE 模型的独特特性使这些取舍可控且可接受。

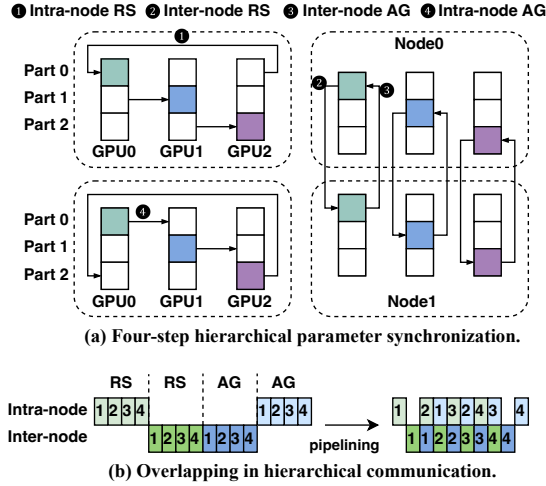


Figure 5. SP 注意力中用于参数同步的层次化通信。

通信效率。使用 TP 时，注意力中的通信量为

$$2bsh(n-1)/n. \quad (1)$$

使用 SP 时，通信量降低为

$$2bsh(n-1)/n \times (2 + 2/m)/n, \quad (2)$$

其中 m 表示 query heads 数量与 key-value heads 数量之比。假设模型在 NVLink 域大小为 8 的 NVIDIA Hopper GPU 工作站上训练，则序列并行注意力的通信延迟可降至张量并行注意力所需延迟的约四分之一。

数据通信与内存开销。SP 注意力与 TP 注意力的一个显著区别在于参数如何分布到设备上：TP 会切分注意力权重，而 SP 会复制这些权重。这会引发一个担忧，即同步梯度和参数的通信开销是否会增加。反直觉的是，考虑到节点内与节点间带宽不对称，以及现代通信库 [35] 采用层次化通信操作（如图 5 所示，并在附录 A.1 中分析），尽管 SP 注意力相比 TP 注意力需要同步 $n \times$ 更多参数，但在实际场景中通信开销差异很小。

另一方面，在 MoE 训练中，SP 注意力引入的额外 GPU 内存消耗很小。对于拥有数十到数百个专家的大规模 MoE 模型，GPU 内存的大部分由专家参数占用。我们在 §6.2 中详述的实验确认，SP 注意力的额外参数同步和内存开销仍然可控。

均衡与不均衡。除了 Ulysses 风格的 SP 注意力，我们也探索了其他形式，包括上下文并行 (CP) [1]，它沿序列维度划分所有激活。然而，由于注意力中的 causal mask，每个 token 只能关注之前的 token，CP 注意力会面临负载不均衡。为缓解该问题，我们尝试了 zigzag 策略，将序列头部和尾部的分区放在同一张 GPU 上，尽管要实现完美均衡仍然困难。因此，在大规模训练中，整个训练过程常常受最不均衡的数据 batch 限制。此外，这种不均衡会扰乱训练流水线，从而降低整体训练效率。

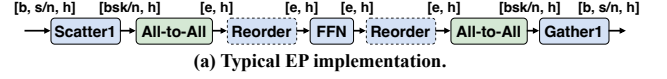


Figure 6. 通信高效的专家并行。e 表示被路由到该 worker 的 token 数量。

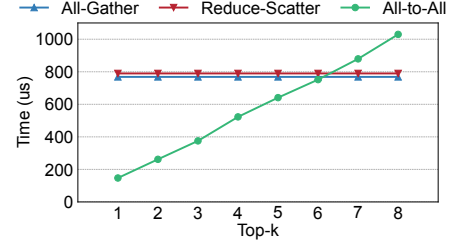


Figure 7. token 分发中 AG、RS 和 A2A 的比较。

3.2 用于前馈网络的专家并行

在前馈网络组件的并行策略选择上，专家并行 (EP) 始终优于张量并行。TP 会切分每个专家的隐藏维度，降低 GEMM 效率；而 EP 在每个设备上保持完整的专家计算。理论上，EP 的通信成本为

$$2k/n \times bsh(n-1)/n, \quad (3)$$

而 TP 的通信成本为

$$2bsh(n-1)/n. \quad (4)$$

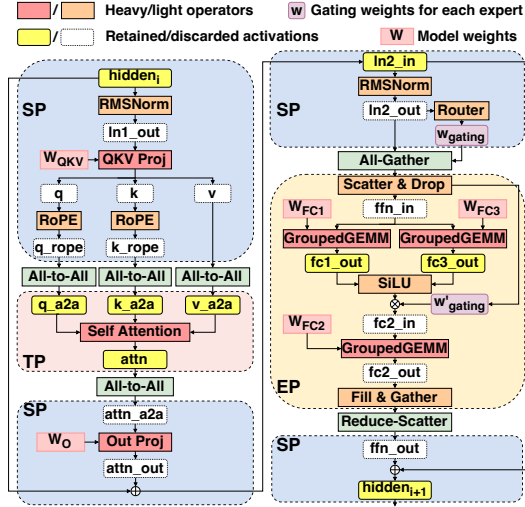
尽管二者的相对效率取决于 k/n 的比值，我们仍针对不同 top-k 值设计自适应通信策略，以最小化 EP 的通信量。

高效通信模式。图 6 比较了典型 EP 实现和 MegaScale-MoE 的方法。标准 EP 实现需要两次 all-to-all 通信，分别用于 token 分发和聚合。此外，在发送前和接收 token 后可能还需要 scatter 操作，以确保分配给同一专家的 token 位于连续内存空间中。

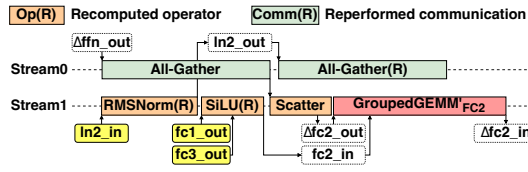
当 top-k 值超过 n 时，我们用 all-gather 和 reduce-scatter 替代传统 all-to-all 通信。首先，all-gather 操作从所有 worker 收集 token。随后，本地 scatter 操作丢弃不需要的 token，只保留当前 worker 上专家所需的 token。专家计算完成后，token 被组装为完整张量。该方法使得通信前可执行 gather 操作，随后通过 reduce-scatter 生成最终结果，确保 EP 的通信开销不高于 TP。

在实际训练中，all-to-all 通信的效率低于 all-gather 和 reduce-scatter，因为它要求每个 worker 与所有其他 worker 通信，而 all-gather 和 reduce-scatter 遵循基于环的通信模式，只与相邻 worker 通信。如图 7 所示，Mixtral-8×7B 中这三种操作的通信时间表明，当 top-k > 6 时，基于 all-gather 的 EP 实现更高效。

高效算子。不同于 Megatron-LM 使用 torch.scatter_add 和 torch.gather 进行张量 scatter 和 gather，我们直接使用 CUDA 开发了高效的 scatter 和 gather 算子。基于 token 路由结果，我们预先计算输入



(a) Forward pass of a MoE layer.



(b) Backward pass snippet with activation rematerialization.

Figure 8. 选择性激活重物化。

张量每一行（代表一个 token）到输出张量对应行的映射。随后，scatter 和 gather 算子按照该映射高效执行数据传输。

负载均衡。MoE 模型训练中的一个众所周知的挑战是专家之间的负载均衡 [22, 26]。为解决该问题，我们使用辅助 loss 和 token dropping 来平衡每个节点内 GPU 之间的工作负载。类似 DeepSeek-V2 [26]，我们将放在同一张 GPU 上的专家视为一组，并按设备而非单个专家计算均衡 loss 和计算容量。

4 通信与计算重叠

在优化并行策略以最小化通信量之后，我们进一步使用全面的通信-计算重叠技术，将通信开销降到近乎为零。训练大模型需要整合多种技术，这增加了通信重叠的复杂度。例如，在任意时刻，设备可能同时处理计算 kernel 和通信 kernel，重叠 PP 与 DP 通信，并管理设备与主机之间的数据传输。Megatron-LM 等现有框架将注意力和 FFN 模块组装为 MoE 层，并依赖 torch.autograd 包进行反向传播，这限制了通信重叠的灵活性。相比之下，MegaScale-MoE 将每个 MoE 层的注意力和 FFN 模块分解为以 GPU kernel 运行的算子，并通过灵活调度实现细粒度通信重叠。

4.1 算子间重叠

我们通过在不同 CUDA stream 上异步执行通信算子和独立计算算子来重叠二者。为了在训练过程中达到最优性能，我们采用了专门手工定制的整体化调度策略。

Activation	Shape	Obtained From
hidden	[b, s/n, h]	# Input
ln1_out	[b, s/n, h]	# RMSNorm(hidden)
qkv	[b, s/n, h(1+2/m)]	# MatMul(ln1_out, qkv_weight)
q_rope	[b, s/n, h]	# RopeEmbedding(q)
k_rope	[b, s/n, h/m]	# RopeEmbedding(k)
qkv_a2a	[b, s, h(1+2/m)/n]	# All-to-All(q_rope, k_rope, v)
attn	[b, s, h/n]	# SelfAttention(qkv_a2a)
attn_a2a	[b, s/n, h]	# All-to-All(attn)
attn_out	[b, s/n, h]	# MatMul(attn_a2a, out_weight)
ln2_in	[b, s/n, h]	# Add(hidden, attn_out)
ln2_out	[b, s/n, h]	# RMSNorm(ln2_in)
ln2_out_ag	[b, s, h]	# All-Gather(ln2_out)
ffn_in	[b*s*k/n, h]	# Scatter(ln2_out_ag)
fc1_out	[b*s*k/n, fh]	# GroupedGEMM(ffn_in, fc1_weight)
fc3_out	[b*s*k/n, fh]	# GroupedGEMM(ffn_in, fc3_weight)
fc2_in	[b*s*k/n, fh]	# SiLU(fc1_out, fc3_out)
fc2_out	[b*s*k/n, h]	# GroupedGEMM(fc2_in, fc2_weight)
fc2_out_rs	[b, s, h]	# Gather(fc2_out)
ffn_out	[b, s/n, h]	# Reduce-Scatter(fc2_out_rs)
hidden(next)	[b, s/n, h]	# Add(ln2_in, ffn_out)

Figure 9. 重物化中的激活形状。

整体化调度。从调用方视角看，我们实现了一个统一的宏模块来执行整个 MoE 层的前向和反向传播，从而扩大调度灵活性。例如，在反向传播期间，可以将各种通信算子与无依赖计算（如激活重算）重叠，以提高效率。从运行时视角看，一个关键挑战是高效管理并发通信任务：需要解决资源冲突以避免阻塞并最大化吞吐。这要求仔细协调，例如确定分配给每个通信算子的 SM 数量，以减少干扰并优化整体吞吐。

选择性激活重物化。整体化调度策略还可以在不损害训练速度的情况下减少内存使用。与计算需求相当的稠密模型相比，MoE 模型由于参数数量大数倍，在训练期间带来显著更高的内存压力。除了使用 ZeRO 优化 [41] 消除 DP 组内冗余优化器状态之外，我们还通过选择性激活重物化进一步优化内存使用。该方法会重新执行那些能够与其他必要算子重叠的计算和通信算子，从而降低激活内存需求。

图 8a 展示了 Mixtral [18] MoE 层的前向传播，并突出标出了该过程中产生的关键激活。MegaScale-MoE 有策略地保留重算代价高的激活，同时重新计算由内存密集型操作或通信操作产生的其他激活。这会最小化对反向计算的依赖，使重物化操作能够与其他计算和通信重叠，避免关键路径延迟。例如，如图 8b 所示，FC2 的 GroupedGEMM 算子在反向传播时需要激活 fc2_in 和 fc2_out 的梯度（记为 $\Delta fc2_out$ ）作为输入。MegaScale-MoE 重算 fc2_in，并将该算子与梯度通信（即针对 Δffn_out 的 all-gather）重叠。类似地，ffn_in 通过重新执行 RMSNorm 和 all-gather 获得，这些算子分别被隐藏在前序通信和 FC2 GroupedGEMM 之中。MegaScale-MoE 还将 ffn_out 的加权求和放在 SwiGLU [45] 激活函数之后立即执行，以消除存储 ffn_out 的需求。这一重排避免了跨越非线性边界的算子，从而保证计算一致性。

图 9 展示了前向传播过程中产生的关键激活形状，其中高亮激活会被保留用于反向传播。设一个 MoE 层内的模型并行规模为 n ，单个专家的中间隐藏大小为 fh 。单个 MoE 层的总激活量为

$$(2n + 2k + 3kf + 12 + 5/m)bsh/n,$$

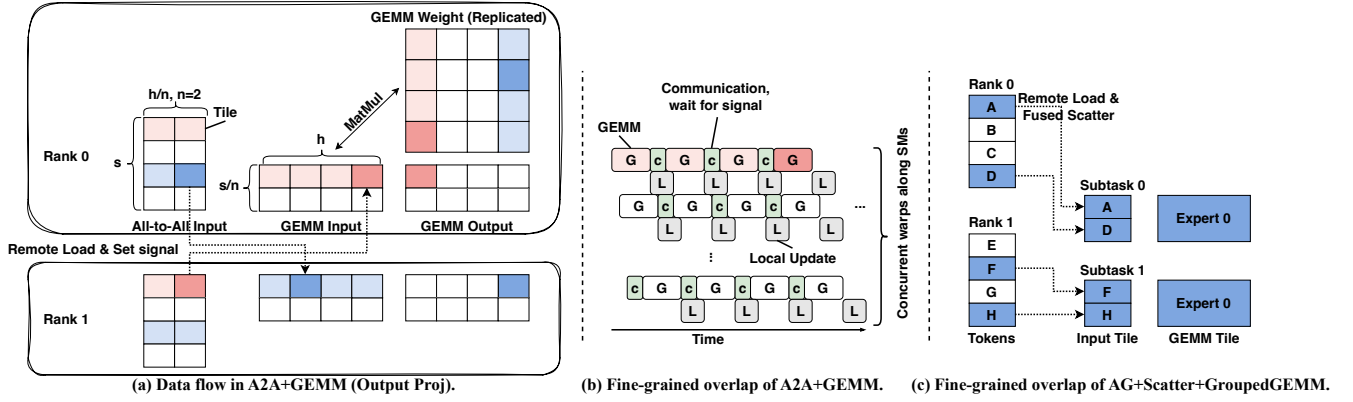


Figure 10. 细粒度算子内通信-计算重叠。

我们将其降低为

$$(2kf + 4 + 2/m)bsh/n.$$

MegaScale-MoE 在保持相同训练速度的同时，将激活内存减少了 ~ 50%。

4.2 算子内重叠

尽管算子间重叠可以有效隐藏通信延迟，但压缩执行时间线中的所有气泡仍非易事，尤其是在前向传播中，没有重物化或梯度计算算子可与通信重叠。一些前向算子直接依赖通信，例如专家计算前的 token 分发；除非引入另一个 micro-batch，否则无法重叠，但这又会增加内存压力。

一种被广泛采用的方案 [19, 50, 52] 是将算子分解为更小的并行算子，并在不同 CUDA stream 上执行，从而实现流水线化。然而，该方法会引入不可忽视的开销：(i) 复杂的 stream 控制，它涉及主机干预，并因 CPU 控制的非确定性而产生随机气泡；(ii) 不完美的尾部计算，会增加整体计算延迟。

为解决上述问题，我们采用算子内重叠，将具有直接依赖关系的通信和计算算子并行化。核心思想是融合这些算子，并将工作负载拆分为 tile。沿用先前工作 [5, 17, 53, 56]，我们在设备内存中实现通信算子与计算算子之间的 barrier。这些 barrier 支持细粒度 tile 级通知，并消除对主机干预的需求，从而进一步提升训练性能。我们分别为注意力模块和 FFN 模块实现了两类 kernel：与 GEMM 重叠的 kernel，以及与 MoE GroupedGEMM 重叠的 kernel。

与 GEMM 重叠。我们首先介绍 GEMM kernel 的算子内通信-计算重叠。具体而言，我们分别为 SP 注意力中的 Output Projection 和 QKV Projection 实现 all-to-all (A2A)+GEMM 以及 GEMM+A2A kernel，其中 X+Y 表示 Y 在 X 之后执行。图 10 展示了 A2A+GEMM 中的数据流和重叠模式。本地数据上的 GEMM 与远端数据通信同时开始。我们利用专用 GPU copy engine 进行数据传输，确保所有 SM (streaming multiprocessors) 都被充分用于计算。一旦远端数据 tile 到达本地内存，一个信号就会

通知 GEMM kernel 继续在已到达的 tile 上计算。对于 GEMM+A2A，all-to-all 操作被融合进 GEMM kernel。每个 GEMM 计算 tile 结束时都会触发一次远端数据传输，将输出数据 tile 写入远端 rank。我们还为张量并行实现了 all-gather+GEMM 和 GEMM+reduce-scatter kernel，它们分别类似于 A2A+GEMM 和 GEMM+A2A。

对于 A2A+GEMM 和 GEMM+A2A，由于 all-to-all 比 all-gather 和 reduce-scatter 更复杂，我们会为通信分配少量 SM。通信使用的 SM 数量经过调优，使通信和计算表现出相近延迟。此外，多个 rank 可能同时从同一设备读取或向同一设备写入，潜在造成 NVLink 竞争。为缓解该问题，我们应用 swizzling [5, 53, 56] 来重排 tile 通信和计算顺序，使通信 tile 的到达节奏与计算 tile 的处理节奏对齐。

与 GroupedGEMM 重叠。对于带有 token 分发和合并的专家并行，我们希望将通信与 GroupedGEMM 重叠。我们实现了两类重叠 kernel：all-gather+scatter+GroupedGEMM 和 GroupedGEMM+gather+reduce-scatter。不同于 GEMM kernel 的重叠技术，MoE GroupedGEMM 需要 token shuffle (scatter/gather)。因此，每个计算 tile 可能来自多个 rank 的 token。为了有效重叠计算与通信，我们对 token 顺序排序，以最小化每个计算 tile 依赖的 rank 数量。此外，由于每个 tile 都有自己的依赖关系，每个 tile 的信号控制会随动态确定的 MoE 路由而变化。

具体而言，对于 AG+scatter+GroupedGEMM，我们根据 token 被路由到的专家索引，沿序列维度重排 token。然后，对每个专家，我们按照源 rank 索引对被路由来的 token 排序。最后，我们将排序后的序列切成多个 block，并使用一系列计算 tile 执行 GroupedGEMM。具体如图 10c 所示，我们基于索引映射选择输入数据行，从而将本地 scatter 融合进 kernel。每个专家的 GroupedGEMM 计算被划分为多个 tile，每个 tile 只依赖一部分源 rank，甚至只依赖单个源 rank。这减少了每个计算 block 的整体等待时间，避免了专家参数的冗余加载，并改善了计算 tile 与通信 tile 之间的重叠。

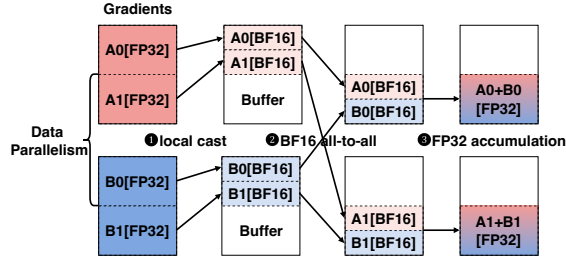


Figure 11. DP 通信压缩。

名称	# 层数	h	#heads	m	h_{ffn}	# 专家	top-k
Internal-352B	60	4096	32	4	14336	32	3
Mixtral-8×7B	32	4096	32	4	14336	8	2
Mixtral-8×22B	56	6144	48	6	16384	8	2
Hunyuan-Large	64	6400	80	10	18304	16	1
Phi-3.5-MoE	32	4096	32	4	6400	16	2
DeepSeekMoE	28	2048	16	1	1408	64	6

Table 2. 评估中的模型配置。

5 通信压缩

我们进一步通过通信压缩来降低通信开销。为了保持收敛稳定性，混合精度训练框架通常会以更高精度（如 FP32）传输等待归约的张量，以确保更精确的累加。数据并行中的梯度 reduce-scatter 就是一个常见例子。

DP 通信压缩。随着 MoE 模型参数增加，数据并行中参数和梯度同步的通信开销也随之增长。已有工作探索了梯度压缩以缓解这一成本。在我们的 BF16 混合精度训练中，我们谨慎地将 FP32 到 BF16 的精度降低用于梯度同步，以平衡效率和收敛稳定性。

具体而言，如图 11 所示，在流水线并行的本地梯度累加过程中，我们将主梯度保留为 FP32。在每个模型阶段完成累加后，我们不再完全依赖 reduce-scatter 进行梯度同步，而是将梯度转换为 BF16，并在数据并行组内执行 all-to-all 通信以收集所需梯度分片，随后在本地以 FP32 聚合。结果表明，与直接使用 FP32 执行 reduce-scatter 相比，该方法引入的精度损失可以忽略不计，同时将梯度通信开销降低 50%。

该方法通过两个关键原因最小化风险。第一，它只在通信期间对累加后的梯度执行一次到 BF16 的转换，而本地梯度累加仍保持 FP32 精度。第二，它没有对 BF16 梯度通信使用环式 reduce，而是采用 all-to-all 通信，并在最终归约中使用 FP32 求和。该设计避免了在基于环的归约中反复累加 BF16 数值可能导致的精度损失。

我们观察到，对大梯度进行类型转换并执行 all-to-all 通信会增加峰值内存消耗，可能导致显存不足。为缓解该问题，我们开发了一个内存高效算子：它将 BF16 梯度原地放入 FP32 输入 buffer 的一半空间，同时将剩余一半作为 BF16 all-to-all 通信的输出 buffer，从而避免峰值内存增长。

FP8 训练的通信压缩。在低精度 FP8 训练中，由于计算时间减少，通信时间占比会增加。为缓解通信开销，我们探索了使用 FP8 精度并结合合适量化技术来压缩通信量。目前，我们在带有张量并行的 FP8 MoE 训练中应用

系统	#GPU	迭代时间 (s)	吞吐 (tokens/s)	训练 1T tokens 所需时间 (天)
Megatron-LM	240	39.94	151.1k	76.61
	480	19.56	301.1k	38.38
	720	13.70	430.5k	26.88
	960	10.82	550.2k	21.23
	1440	7.90	746.6k	15.50
MegaScale-MoE	240	21.61	272.9k (1.81×)	42.41
	480	11.83	498.6k (1.65×)	23.21
	720	7.97	740.1k (1.72×)	15.64
	960	6.12	963.8k (1.77×)	12.01
	1440	4.19	1407.7k (1.88×)	8.22

Table 3. 使用 NVIDIA H800 GPU 训练 352B MoE 模型的强扩展性能。吞吐列括号中的数字表示 MegaScale-MoE 相比 Megatron-LM 的加速比。

通信压缩，重点处理容易发生上溢或下溢的归约场景。例如，我们对所有张量采用 E4M3 格式（4 位指数、3 位尾数）。类似 DP reduce-scatter 压缩，我们在前向传播中用 FP8 all-to-all 替代 BF16 TP reduce-scatter，并以 FP32 精度执行归约。在对应的反向传播中，我们对梯度应用 FP8 all-gather。值得注意的是，简单降低精度会导致 loss 与 BF16 训练不对齐。为缓解该问题，我们对前向通信采用按 token 的激活量化，对反向通信采用按 channel 的量化。在反向传播中，我们进一步沿 token 维度使用较小 group size（例如 128）进行分组量化。

6 评估

本节对 MegaScale-MoE 进行全面评估，覆盖整体训练性能 (§6.1)、MegaScale-MoE 关键优化的消融研究 (§6.2)，以及精度-通信协同设计的有效性 (§6.3)。表 2 列出了评估中使用的 MoE 模型配置，包括隐藏维度 (h)、FFN 中间维度 (h_{ffn})、专家数量以及 top-k 值。除非另有说明，评估均在 NVIDIA H800 GPU 上进行，其规格见表 4。

6.1 训练性能

MegaScale-MoE 构建在 Megatron-LM [48] 之上。Megatron-LM 是最先进的开源 LLM 训练系统之一，支持 3D 并行策略，并持续吸收社区最新优化。我们的评估使用 GitHub 上 commit hash 为 f1f03922 的 Megatron-LM [32]，选择该版本是因为它在数月前实验开始时较为稳定。为公平比较，我们对 Megatron-LM 和 MegaScale-MoE 使用相同的全局 batch size，并分别为两个系统选择最优并行配置。具体而言，MegaScale-MoE 在每个节点内采用 SP attention 和 EP，而 Megatron-LM 在每个节点内采用 TP；两个系统都配置 PP size 为 15。我们调优 Megatron-LM 的配置，以满足其要求所有组件使用统一 TP size 的限制。如 §3.1 所讨论，对于 Megatron-LM，TP size 为 1 会导致无法接受的 8× 激活内存（只能通过缓慢的 gradient checkpointing 重算来处理），而 TP size 为 8 又会迫使 EP 跨节点运行，产生比 PP 更高的通信成本。值得注意的是，评估中的两个系统都启用了 MegaScale [19] 中用于数据并行和流水线并行的通信-计算重叠技术。因此，通信开销主要来自节

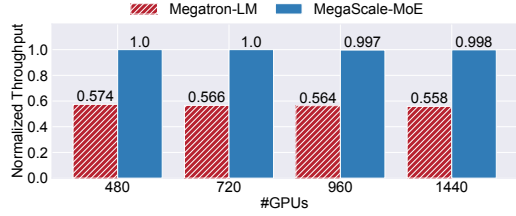


Figure 12. 使用 NVIDIA H800 GPU 训练 352B MoE 模型的弱扩展性能。

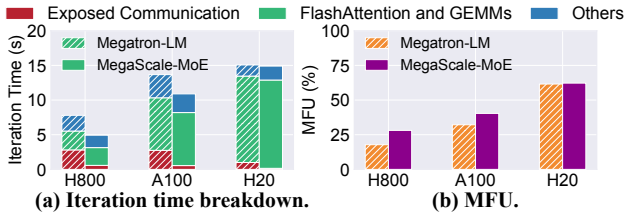


Figure 13. 在不同 GPU 上训练 Mixtral-8×7B 的性能分解。

GPU	计算能力 (TFLOPS)	内存规格		NVLink Bw. (GB/s)
		容量 (GB)	带宽 (TB/s)	
H800	989	80	3.4	400
A100	312	80	2.0	600
H20	148	96	4.0	900

Table 4. 不同 NVIDIA GPU 的规格。

点内模型并行，例如 TP、SP 和 EP。序列长度为 8,192，词表大小为 65,536。

可扩展性。表 3 比较了 Megatron-LM 和 MegaScale-MoE 在 352B MoE 模型上的强扩展训练性能。我们在保持全局 batch size 固定为 720 的情况下扩展 GPU 数量。在所有设置下，MegaScale-MoE 相比 Megatron-LM 实现 1.65–1.88× 加速。随着 GPU 数量增加，MegaScale-MoE 的 MFU (Model FLOPs Utilization) 从 32.48% 下降到 27.89%。这是预期现象，因为 batch size 固定，而随着 GPU 增多，每条流水线的 micro-batch 数量减少，从而产生更多气泡。

图 12 展示了 Megatron-LM 和 MegaScale-MoE 在同一模型上的弱扩展训练性能。我们按 GPU 数量（从 480 到 1,440）等比例将全局 batch size 从 360 扩展到 1,080。相比 Megatron-LM，MegaScale-MoE 实现了 1.74–1.79× 的训练吞吐。随着规模增大，Megatron-LM 因通信开销增加而吞吐下降 2.74%。相比之下，MegaScale-MoE 受益于全面的通信-计算重叠，表现出近线性扩展性，吞吐仅下降 0.2%。

不同 GPU 上的性能分解。为进一步理解生产环境中训练 MoE 模型的性能，我们对 MegaScale-MoE 进行了深入分析。我们分别在 32 张 NVIDIA H800、H20 和 A100 GPU 上训练 Mixtral-8×7B。所用 GPU 的规格列于表 4。我们将 DP size 设为 4，Megatron-LM 的 TP size 设为 8，MegaScale-MoE 的 SP 和 EP size 设为 8。如图 13b 所示，在这些 GPU 上，MegaScale-MoE 在 MFU 上始终优于 Megatron-LM，最高达到 1.58×。图 13a 展示了 Megatron-LM 和 MegaScale-MoE 的迭代时间分解。暴露通信时间表示未与计算操作

编号	方法	归一化吞吐	Δ
1	baseline	1	
2	(1) 加 SP+EP	1.13	+13%
3	(2) 加算子间重叠	1.22	+9%
4	(3) 加算子内重叠	1.28	+6%

Table 5. 使用 240 张 NVIDIA H800 GPU、batch size 为 720 训练 352B MoE 模型时的吞吐提升分解。

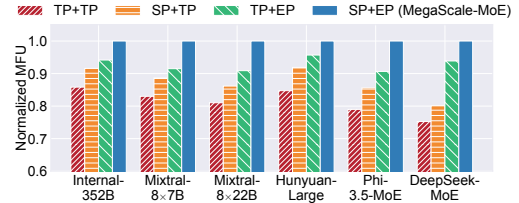


Figure 14. 不同模型的并行效率。

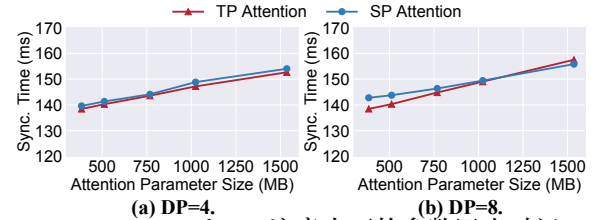


Figure 15. SP 和 TP 注意力下的参数同步时间。

重叠的通信时间。FlashAttention 和 GEMM 是我们计算 MFU 时统计的操作。性能提升主要来自 MegaScale-MoE 的通信高效并行策略和细粒度重叠通信。

需要注意，随着 GPU 计算能力增强，MFU 数值会下降。这是因为不同于稠密模型，MoE 模型包含许多内存密集型操作，如 routing、本地 scatter 和 gather；由于内存带宽并未像计算能力一样快速增长，这些操作仍然耗时。此外，GEMM 效率也会随计算能力提高而下降，因为它同样依赖受内存带宽约束的内存加载。

6.2 消融研究

我们评估 MegaScale-MoE 各项优化技术的有效性。首先，我们通过逐步启用每项技术进行系统性分解实验，以隔离其对整体性能的贡献。表 5 展示了在 240 张 GPU 上以全局 batch size 720 训练 352B MoE 模型时，不同优化带来的吞吐提升分解。baseline 是 MegaScale-MoE 的一个版本，它对注意力和 FFN 都采用 TP，并禁用通信-计算重叠。首先，通过应用通信高效策略，即对注意力采用 SP、对专家采用 EP，我们相对该 baseline 获得初始 13% 的吞吐提升。随后，我们针对大规模 MoE 训练中的主要瓶颈，即通信开销。我们的算子间和算子内重叠方法有效隐藏这些成本，分别进一步带来额外 9% 和 6% 的训练加速。

在系统性分解之后，我们对每个组件进行消融研究。每次只改变一个设置，并保持其他设置不变，以更深入理解其行为。

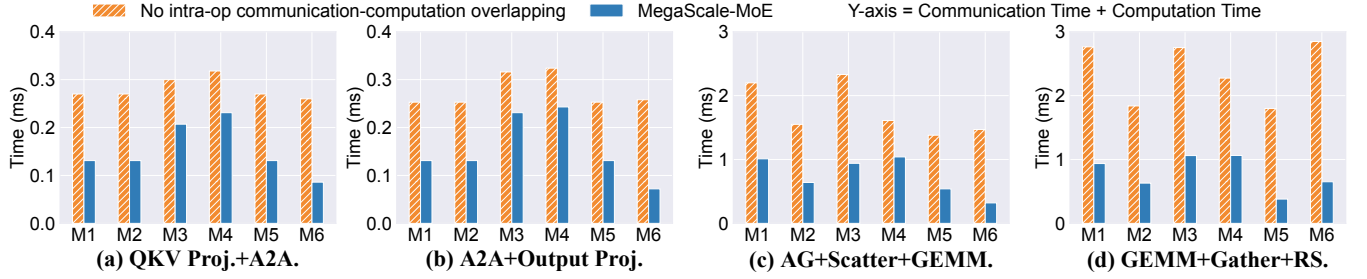


Figure 16. 每层中重叠后的通信-计算时间与未重叠时间对比。M1-M6 表示表 2 中自上而下列出的六个模型；A2A、AG 和 RS 分别表示 all-to-all、all-gather 和 reduce-scatter。

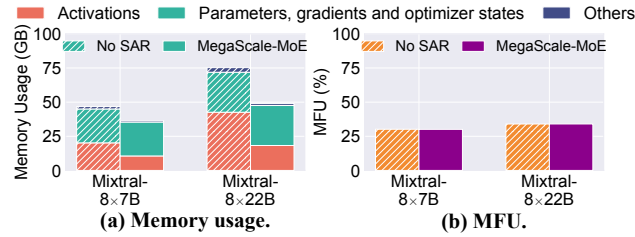


Figure 17. 选择性激活重物化 (SAR) 的消融研究。

并行策略。我们使用一个包含 8 张 NVIDIA H800-SXM GPU 的单节点，比较多种节点内并行策略下的训练效率。我们将并行策略记为 X+Y，其中 X 表示注意力的并行策略，Y 表示专家的并行策略。注意力可用的并行策略包括 TP 和我们的 SP，而专家可选 TP 和 EP。为了隔离优化并行带来的性能收益，我们禁用其他系统优化。

我们测量了一个内部 MoE 模型和五个开源 MoE 模型的训练 MFU；这些模型具有表 2 所列的不同配置。全局 batch size 设为 32，并且我们调整每个模型的层数以适配 GPU 内存。图 14 表明，MegaScale-MoE 的并行策略 SP+EP 始终优于其他三种并行策略，相比 TP+TP 获得 14.9%–32.9% 更高 MFU。性能收益归因于两个主要因素。第一，如 §3 所讨论，相比 TP，SP 和 EP 有效减少通信量，从而降低通信开销。第二，TP 沿中间维度切分 FFN 模块，导致 GEMM 效率更低。

为了更全面地评估并行策略，我们还报告了 SP 中复制注意力参数引入的额外开销。在内存使用方面，相比 TP，SP 在全部七个模型上带来 1.2%–5.4% 更高的内存占用，并需要多 1.7%–8.1% 的内存来存储参数、梯度和优化器状态。考虑到 SP 带来的显著性能收益，这一开销是可控的。

对于参数同步时间，我们遵循大规模训练设置，将 TP 或 SP 的规模设为 8，从而在单节点内有效并行化每一层。每张 GPU 上的注意力参数大小从 384 MB 变化到 1536 MB，而 FFN 参数大小固定为每张 GPU 10 GB，以反映典型真实训练设置。我们使用 SP 和 TP attention 运行 MegaScale-MoE，并使用 4 个和 8 个 DP 组，分别对应总计 32 张和 64 张 GPU。图 15 表明，SP 和 TP attention 的同步时间始终相当，差异仅为 0.3%–3.1%。这与我们的假设一致，即 SP 和 TP 在 DP 通信延迟方面具有相似性能特征。

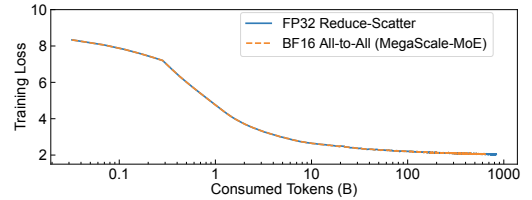


Figure 18. 启用 DP 通信压缩时 MegaScale-MoE 的训练 loss 曲线。

算子内通信重叠。接着，我们测量前向传播中四组关键通信及其对应计算算子的耗时：(i) QKV Projection 与 all-to-all，(ii) all-to-all 与 Output Projection，(iii) all-gather 与 scatter 和 GroupedGEMM，(iv) GroupedGEMM 与 gather 和 reduce-scatter，如图 8 所示。图 16 表明，在全部六个模型上，相比缺少细粒度重叠的 baseline，MegaScale-MoE 将通信和计算算子的总时间降低了 1.2–4.7 $\times$ 。由于算子内通信-计算重叠，MegaScale-MoE 将训练迭代时间减少了 7.1%–12.9%。

选择性激活重物化。我们将 MegaScale-MoE 与禁用选择性激活重物化 (No SAR) 的 baseline 进行比较；该 baseline 在训练期间将所有激活都存储在 GPU 内存中。我们在 128 张 NVIDIA H800 GPU 上训练 Mixtral-8x7B 和 Mixtral-8x22B 来评估两种方法。图 17 展示了内存使用分解和训练 MFU。相比 No SAR，MegaScale-MoE 分别为两个模型将激活内存消耗降低 45.5% 和 57.2%，带来 21.3% 和 35% 的整体内存降低，同时将训练性能差异保持在 0.5% 以内。

数据并行通信压缩。如 §5 所述，我们通过使用 BF16 all-to-all DP 通信和 FP32 reduce-scatter 通信训练一个 7B MoE 模型，验证通信压缩技术的有效性。图 18 展示了几乎完全一致的训练 loss 曲线。该优化只压缩 batch 累加后的梯度，并且只在通信期间执行 BF16 与 FP32 之间的转换，因此风险很小。

6.3 模型收敛

我们评估 MegaScale-MoE 的模型收敛性。图 19 展示了从头训练 35B MoE 模型以及从 checkpoint 继续训练 176B MoE 模型的 loss 曲线，并给出了 BF16 和 FP8 两种精度的结果。MegaScale-MoE 在 BF16 和 FP8 格式下都能保证稳定收敛和一致的训练 loss。

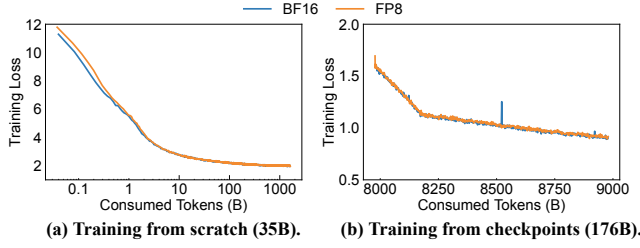


Figure 19. MegaScale-MoE 在 FP8 和 BF16 下的 loss 曲线。

7 经验

本节描述我们部署和运维 MegaScale-MoE 的经验。

部署经验。 MegaScale-MoE 已部署在我们的生产环境中，并负责公司内部大多数大规模 MoE 训练任务。它支持训练拥有万亿参数的模型，支持单个训练作业扩展到超过 10,000 张 GPU，且单个训练任务可运行数月。通过结合前述技术，MegaScale-MoE 在不损害模型性能的情况下，最小化 MoE 训练中的空闲通信时间并优化内存使用，最终在大规模 MoE 训练中节省数百万 GPU 小时。图 20 展示了一个真实生产作业中的模型收敛情况。该作业训练一个专有 MoE 模型，总参数量为 200B，每个 token 激活 20B 参数。该作业使用超过 10,000 张 GPU，持续数月。loss 在稳定训练过程中持续收敛。

FP8 训练。 为保持 FP8 训练的收敛稳定性，我们做了大量工作。例如，我们观察到 SwiGLU 算子会显著扩大数值范围。为解决该问题，我们用精度更高的按 token 量化 ($1 \times h$) 替代按 tensor 量化。此外，由于 SwiGLU 与 gating weight 相乘会进一步放大动态数值范围，我们将 gating weight 乘法后移到 FC2 输出之后，从而减少量化误差。

除了确保训练收敛，我们还引入了额外工程优化。现有 FP8 训练实现 [25, 50] 以 BF16 存储模型参数，因此在 GEMM 计算前需要频繁转换到 FP8，增加了类型转换和转置开销。为解决该问题，我们使用多精度优化器，直接以 FP8 存储模型参数，同时将主参数保留为 FP32，并为不同数据类型使用独立 buffer。这降低了内存消耗，并将数据并行中的参数 all-gather 通信减半。

扩展规模。 在训练 MoE 模型时，一个有趣的工程问题是：能否在不增加计算负载的情况下，通过增加模型参数无限扩展训练规模？在张量并行中，这种方法并不实际，因为扩大模型需要更高 TP degree 来容纳额外参数。虽然增加 TP 会减少每张 GPU 上的计算，但如式 1 和 4 所示，通信开销保持不变，导致通信时间逐渐变长并降低训练效率。换言之，TP 存在固有可扩展性限制，并且常依赖高速节点内链路来缓解通信延迟。

相比之下，当使用 SP 和 EP 扩展训练时，如式 2 和 3 所示，通信量会随着并行规模 n 增大而下降。这意味着从理论上讲，该并行策略可以扩展到显著更大的规模。然而，在实际层次化基础设施中，一个关键挑战随之出现：当扩展超出 NVLink 域、带宽下降到 RDMA 水平时，该方法能否保持训练效率？

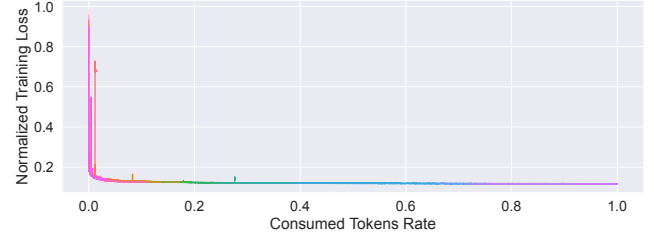


Figure 20. 一个真实生产作业的归一化训练 loss 曲线。该作业在超过 10,000 张 GPU 上运行数月，在数万亿 token 上训练一个每次激活 20B 参数、总参数量 200B 的 MoE 模型。不同颜色表示训练重启。

形式化地，对于包含 MoE 机制的 SwiGLU 结构，计算时间与通信时间的比值 R 定义为：

$$\text{comm_time} = \frac{2k \times bsh(n-1)/n/n}{\text{bandwidth}}, \quad (5)$$

$$\text{comp_time} = \frac{3k \times bsh \times h_{ffn}/n}{\text{peak}}. \quad (6)$$

$$R = \frac{\text{comp_time}}{\text{comm_time}} \quad (7)$$

$$= 3/2 \times h_{ffn} \times \frac{\text{bandwidth}}{\text{peak}} \times n/(n-1) \quad (8)$$

$$\approx 3/2 \times h_{ffn} \times \frac{\text{bandwidth}}{\text{peak}} \quad (9)$$

为了维持训练效率，FFN 的计算时间必须超过通信时间，以确保通信开销能够被有效重叠。因此，我们的目标是保持 $R > 1$ ，这带来两个关键洞察：

- R 的值与专家数量、top-k、隐藏维度、并行规模或输入大小无关，因此算法参数选择具有灵活性。
- R 只由专家的中间维度、计算峰值和通信带宽决定。因此，在固定硬件上，只要专家维度足够大，从工程角度看，MoE 模型就可以在保持训练效率的同时继续扩展。

整体化与自动化。 我们在算子间通信-计算重叠上投入了大量工程工作，包括确定算子执行顺序、通信与计算的并发方式，以及通信的 SM 分配。这些人工干预帮助我们更深入理解训练动态，并支持有针对性的优化。随着训练推进和经验积累，我们希望在搜索空间内自动化算子调度，以细粒度优化训练过程并达到最优性能。自动优化留作未来工作。

MoE 与稠密模型训练。 在持续优化 MoE 模型训练的过程中，我们识别出它与稠密模型训练之间的几个关键差异。在稠密 Transformer 层中，优化工作集中在自注意力和 GEMM 上。前者通常由 FlashAttention [8] 等技术加速，而后者作为稠密计算，一般可以在 GPU 并行处理单元上达到较高利用率。相比之下，如图 13a 所示，注意力和 GroupedGEMM 的总运行时间只占一层执行时间的大约三分之一。其余时间由通信和其他算子消耗。尽管 MegaScale-MoE 有效处理了通信开销，我们观察到 MoE 模型中的计算算子本身比稠密模型中的对应算子更复

杂，也会引入性能下降。具体而言，它们是 straggler 的主要来源，原因有三点：

第一，每个专家的中间维度小于稠密模型中的 FFN 层。为了高效并发处理多个专家的计算，GroupedGEMM 使用单个 CUDA kernel 执行大量小矩阵乘法。该 kernel 的资源使用，包括 shared memory、L1 cache 和线程数量，会通过 cuFuncSetAttribute 进行细粒度控制。然而，这种细粒度控制可能引入同步延迟。第二，由于路由到每个专家的 token 数量不均衡，GroupedGEMM 的输入和输出是动态形状张量。这些张量的频繁分配和释放会加剧 GPU 内存碎片。第三，MoE gating 机制涉及大量小算子，用于计算路由分数、通信路由决策等任务。CPU 性能抖动可能延迟这些 kernel 的启动，以至于启动延迟超过它们在 GPU 上的实际执行时间，从而产生流水线气泡。

8 相关工作

大模型训练。LLM 研究推动了可扩展、高效且鲁棒的训练技术发展 [19, 43, 48, 54]，以满足这些模型巨大的计算需求。DeepSpeed [43] 提供 Zero Redundancy Optimizer (ZeRO) [41, 42, 44]，它在数据并行中将模型参数、梯度和优化器状态切分到参与 GPU 上，使 LLM 能够以可控内存消耗扩展。Megatron-LM [48] 聚焦层内模型并行技术，切分每一层的参数和计算。流水线并行将连续一组层的参数和计算分配给每张 GPU [14, 33]，将一个 batch 拆成 micro-batch，并以流水线方式处理这些 micro-batch。MegaScale [19] 展示了结合张量、流水线和数据并行如何成为一种高效策略，用以前所未有的规模训练数十亿参数以上的大模型。

专家混合训练。为应对训练先进神经网络的计算挑战，机器学习领域越来越多采用专家混合架构。随后，许多深度学习框架被提出，用于在多 GPU 集群上训练或推理 MoE。DeepSpeed-MoE [40] 通过模型架构设计和压缩技术显著降低训练成本。HetuMoE [36] 利用层次化 all-to-all 通信策略实现性能加速。SE-MoE [47] 聚焦使用 CPU 内存和 SSD 等异构资源进行可扩展高效训练。FasterMoE [13] 引入一整套优化，如动态 shadowing、细粒度调度，以及避免阻塞的专家选择策略。Janus [30] 为 MoE 模型提出以数据为中心的范式转变，目标是降低通信需求并提升训练效率。Tutel [15] 为 MoE 模型提供动态方案，采用自适应并行和流水线。然而，对于拥有数千亿参数的模型，其动态并行切换和层次化 all-to-all 可能造成显著开销。为避免这类开销，最新 MoE 训练系统 [26, 27] 使用辅助 loss 或 routing bias 进行负载均衡，并限制跨节点 token 分发。通过将每个 MoE 层映射到节点内，MegaScale-MoE 消除了跨节点 token 分发。

近期，DeepSeek-V3 [27] 为生产规模 MoE 模型训练引入了两个关键优化：DeepEP 用于高性能跨节点 all-to-all 通信，DualPipe 用于重叠通信与计算。由于跨节点 InfiniBand 带宽相对较低，DeepEP 将 token 分发限制在最多 4 个节点内，以保持恒定跨节点通信量，但这限制了其路由灵活性。相比之下，MegaScale-MoE 将每个 MoE 层放置在节点内，以保证可高效路由到任意 top-k

专家。DualPipe 利用流水线并行在不同 micro-batch 之间进行通信-计算重叠，这需要存储 $2\times$ 的模型参数。相比之下，MegaScale-MoE 的重叠发生在单个 micro-batch 的前向或反向传播内部，不引入额外内存开销，并且同时兼容带有和不带有流水线并行的系统。

长上下文训练。Megatron-LM [20, 48] 选择只沿序列维度划分特定操作，而用于训练长上下文模型的多种序列并行方法 [11, 21, 23, 29] 已被探索。Blockwise Parallel Transformer [28] 基于 online softmax 计算，实现自注意力的分块计算和 FFN 融合。Ring Attention [23, 29] 引入与自注意力计算集成的环式通信机制，以促进 key 和 value chunk 的交换。我们采用 DeepSpeed Ulysses [16] 中 all-to-all 风格的 SP attention。由于它按 head 而不是序列长度划分注意力，因此通信量更低且计算模式更均衡。

通信-计算重叠。一些框架 [12, 24, 31, 39, 55] 聚焦于在采用单一并行策略的分布式深度学习训练中重叠通信与计算。一些编译器风格工作 [17, 37, 52] 提供 kernel 间细粒度重叠，但过度切分 GEMM kernel 可能导致 GPU 利用率较低。Centauri [6] 通过通信划分和层次化调度，增强采用 3D 并行的 LLM 训练中的通信重叠。类似 Centauri，我们的算子间通信重叠通过重排算子，将通信隐藏在独立计算之中。我们进一步通过算子内重叠隐藏关键路径上的通信，同时不损害 GPU 利用率。

9 结论

本文深入介绍了 MegaScale-MoE 的设计、实现和部署。它是一个生产级系统，用于高效训练 MoE 模型。MegaScale-MoE 利用通信高效方法，包括通信量更低的并行策略、算子间和算子内通信-计算重叠，以及结合调整后通信模式的通信压缩，从而释放高性能 GPU 的计算能力。在 1,440 张 NVIDIA Hopper GPU 上训练 352B MoE 模型时，MegaScale-MoE 达到 1.41M tokens/s 吞吐，相比 Megatron-LM 提升 1.88 $\times$ 。通过分享我们关于加速大规模 MoE 训练的见解，我们希望本工作能够启发未来研究。

致谢。我们感谢论文指导委员 Cheng Li 以及匿名审稿人提供的宝贵反馈和建议。本工作部分受国家重点研发计划项目（2022YFB4500700）、青年教师科研创新能力支持项目（ZYGXQNJSKYCXNLZCXM-II）、北京大学中央高校基本科研业务费，以及国家自然科学基金项目（62172008、62325201）资助。金鑫和刘鑫为通讯作者。金超、刘讚哲和金鑫亦隶属于教育部高可信软件技术重点实验室（北京大学）。

参考文献

- [1] 2025. Context parallelism in Megatron-LM. (2025). https://docs.nvidia.com/megatron-core/developer-guide/latest/api-guide/context_parallel.html.
- [2] 2025. Introducing DBRX: A New State-of-the-Art Open LLM. (2025). <https://www.databricks.com/blog/introducing-dbrx-new-state-art-open-llm>
- [3] 2025. Open Release of Grok-1. (2025). <https://x.ai/blog/grok-os>
- [4] Joshua Ainslie, James Lee-Thorp, Michiel de Jong, Yury Zemlyanskiy, Federico Lebrón, and Sumit Sanghai. 2023. Gqa: Training generalized

- multi-query transformer models from multi-head checkpoints. *arXiv preprint arXiv:2305.13245* (2023).
- [5] Li-Wen Chang, Wenlei Bao, Qi Hou, Chengquan Jiang, Ningxin Zheng, Yinmin Zhong, Xuanrun Zhang, Zuquan Song, Chengji Yao, Ziheng Jiang, et al. 2024. FLUX: fast software-based communication overlap on gpus through kernel fusion. *arXiv preprint arXiv:2406.06858* (2024).
 - [6] Chang Chen, Xiuhong Li, Qianchao Zhu, Jiangfei Duan, Peng Sun, Xingcheng Zhang, and Chao Yang. 2024. Centauri: Enabling Efficient Scheduling for Communication-Computation Overlap in Large Model Training via Communication Partitioning. In *ACM ASPLOS*.
 - [7] Aakanksha Chowdhery, Sharan Narang, Jacob Devlin, Maarten Bosma, Gaurav Mishra, Adam Roberts, Paul Barham, Hyung Won Chung, Charles Sutton, Sebastian Gehrmann, et al. 2023. Palm: Scaling language modeling with pathways. *Journal of Machine Learning Research* (2023).
 - [8] Tri Dao, Dan Fu, Stefano Ermon, Atri Rudra, and Christopher Ré. 2022. Flashattention: Fast and memory-efficient exact attention with io-awareness. *Neural Information Processing Systems* (2022).
 - [9] Nan Du, Yanping Huang, Andrew M Dai, Simon Tong, Dmitry Lepikhin, Yuanzhong Xu, Maxim Krikun, Yanqi Zhou, Adams Wei Yu, Orhan Firat, Barret Zoph, Liam Fedus, Maarten P Bosma, Zongwei Zhou, Tao Wang, Emma Wang, Kellie Webster, Marie Pellat, Kevin Robinson, Kathleen Meier-Hellstern, Toju Duke, Lucas Dixon, Kun Zhang, Quoc Le, Yonghui Wu, Zhifeng Chen, and Claire Cui. 2022. GLaM: Efficient Scaling of Language Models with Mixture-of-Experts. In *International Conference on Machine Learning (ICML)*.
 - [10] William Fedus, Barret Zoph, and Noam Shazeer. 2022. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *Journal of Machine Learning Research* (2022).
 - [11] Diandian Gu, Peng Sun, Qinghao Hu, Ting Huang, Xun Chen, Ying-tong Xiong, Guoteng Wang, Qiaoling Chen, Shangchun Zhao, Jiarui Fang, et al. 2024. Loongtrain: Efficient training of long-sequence llms with head-context parallelism. *arXiv preprint arXiv:2406.18485* (2024).
 - [12] Sayed Hadi Hashemi, Sangeetha Abdu Jyothi, and Roy Campbell. 2019. Tictac: Accelerating distributed deep learning with communication scheduling. *Proceedings of Machine Learning and Systems* (2019).
 - [13] Jiaao He, Jidong Zhai, Tiago Antunes, Haojie Wang, Fuwen Luo, Shangfeng Shi, and Qin Li. 2022. Fastermoe: modeling and optimizing training of large-scale dynamic pre-trained models. In *ACM PPoPP*.
 - [14] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. 2019. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Neural Information Processing Systems* (2019).
 - [15] Changho Hwang, Wei Cui, Yifan Xiong, Ziyue Yang, Ze Liu, Han Hu, Zilong Wang, Rafael Salas, Jithin Jose, Prabhat Ram, et al. 2023. Tutel: Adaptive mixture-of-experts at scale. *Proceedings of Machine Learning and Systems* (2023).
 - [16] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. 2023. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509* (2023).
 - [17] Abhinav Jangda, Jun Huang, Guodong Liu, Amir Hossein Nodehi Sabet, Saeed Maleki, Youshan Miao, Madanlal Musuvathi, Todd Mytkowicz, and Olli Saarikivi. 2022. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *ACM ASPLOS*.
 - [18] Albert Q Jiang, Alexandre Sablayrolles, Antoine Roux, Arthur Mensch, Blanche Savary, Chris Bamford, Devendra Singh Chaplot, Diego de las Casas, Emma Bou Hanna, Florian Bressand, et al. 2024. Mixtral of experts. *arXiv preprint arXiv:2401.04088* (2024).
 - [19] Ziheng Jiang, Haibin Lin, Yinmin Zhong, Qi Huang, Yangrui Chen, Zhi Zhang, Yanghua Peng, Xiang Li, Cong Xie, Shibiao Nong, Yulu Jia, Sun He, Hongmin Chen, Zhihao Bai, Qi Hou, Shipeng Yan, Ding Zhou, Yiyao Sheng, Zhuo Jiang, Haohan Xu, Haoran Wei, Zhang Zhang, Pengfei Nie, Leqi Zou, Sida Zhao, Liang Xiang, Zherui Liu, Zhe Li, Xiaoying Jia, Jianxi Ye, Xin Jin, and Xin Liu. 2024. MegaScale: Scaling Large Language Model Training to More Than 10,000 GPUs. In *USENIX NSDI*.
 - [20] Vijay Anand Korthikanti, Jared Casper, Sangkug Lym, Lawrence McAfee, Michael Andersch, Mohammad Shoeybi, and Bryan Catanzaro. 2023. Reducing activation recomputation in large transformer models. *Proceedings of Machine Learning and Systems* (2023).
 - [21] Dacheng Li, Rulin Shao, Anze Xie, Eric P. Xing, Xuezhe Ma, Ion Stoica, Joseph E. Gonzalez, and Hao Zhang. 2024. DISTFLASHATTN: Distributed Memory-efficient Attention for Long-context LLMs Training. *arXiv preprint arXiv:2310.03294* (2024).
 - [22] Jiamin Li, Yimin Jiang, Yibo Zhu, Cong Wang, and Hong Xu. 2023. Accelerating distributed {MoE} training and inference with lina. In *USENIX ATC*.
 - [23] Shenggui Li, Fuzhao Xue, Chaitanya Baranwal, Yongbin Li, and Yang You. 2021. Sequence parallelism: Long sequence training from system perspective. *arXiv preprint arXiv:2105.13120* (2021).
 - [24] Shen Li, Yanli Zhao, Rohan Varma, Omkar Salpekar, Pieter Noordhuis, Teng Li, Adam Paszke, Jeff Smith, Brian Vaughan, Pritam Damania, et al. 2020. Pytorch distributed: Experiences on accelerating data parallel training. *arXiv preprint arXiv:2006.15704* (2020).
 - [25] Wanchao Liang, Tianyu Liu, Less Wright, Will Constable, Andrew Gu, Chien-Chin Huang, Iris Zhang, Wei Feng, Howard Huang, Junjie Wang, et al. 2024. TorchTitan: One-stop PyTorch native solution for production ready LLM pre-training. *arXiv preprint arXiv:2410.06511* (2024).
 - [26] Aixiu Liu, Bei Feng, Bin Wang, Bingxuan Wang, Bo Liu, Chenggang Zhao, Chengqi Deng, Chong Ruan, Damai Dai, Daya Guo, et al. 2024. Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. *arXiv preprint arXiv:2405.04434* (2024).
 - [27] Aixiu Liu, Bei Feng, Bing Xue, Bingxuan Wang, Bochao Wu, Chengda Lu, Chenggang Zhao, Chengqi Deng, Chenyu Zhang, Chong Ruan, et al. 2024. Deepseek-v3 technical report. *arXiv preprint arXiv:2412.19437* (2024).
 - [28] Hao Liu and Pieter Abbeel. 2024. Blockwise Parallel Transformers for Large Context Models. *Neural Information Processing Systems* (2024).
 - [29] Hao Liu, Matei Zaharia, and Pieter Abbeel. 2023. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889* (2023).
 - [30] Juncai Liu, Jessie Hui Wang, and Yimin Jiang. 2023. Janus: A unified distributed training framework for sparse mixture-of-experts models. In *ACM SIGCOMM*.
 - [31] Kshiteej Mahajan, Ching-Hsiang Chu, Srinivas Sridharan, and Aditya Akella. 2023. Better Together: Jointly Optimizing ML Collective Scheduling and Execution Planning using {SYNDICATE}. In *USENIX NSDI*.
 - [32] Megatron-LM 2025. GPU optimized techniques for training transformer models at-scale. (2025). <https://github.com/NVIDIA/Megatron-LM>.
 - [33] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Shadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. 2019. PipeDream: generalized pipeline parallelism for DNN training. In *ACM SOSP*.
 - [34] Deepak Narayanan, Mohammad Shoeybi, Jared Casper, Patrick LeGresley, Mostofa Patwary, Vijay Korthikanti, Dmitri Vainbrand, Prethvi Kashinkunti, Julie Bernauer, Bryan Catanzaro, et al. 2021. Efficient large-scale language model training on gpu clusters using megatron-lm. In *International Conference for High Performance Computing, Networking, Storage and Analysis*.

- [35] NCCL 2025. Optimized primitives for inter-GPU communication. (2025). <https://github.com/NVIDIA/nvcl>.
- [36] Xiaonan Nie, Pinxue Zhao, Xupeng Miao, Tong Zhao, and Bin Cui. 2022. HetuMoE: An efficient trillion-scale mixture-of-expert distributed training system. *arXiv preprint arXiv:2203.14685* (2022).
- [37] Suchita Pati, Shaizeen Aga, Mahzabeen Islam, Nuwan Jayasena, and Matthew D. Sinclair. 2024. T3: Transparent Tracking & Triggering for Fine-grained Overlap of Compute & Collectives. In *ACM ASPLOS*.
- [38] Houwen Peng, Kan Wu, Yixuan Wei, Guoshuai Zhao, Yuxiang Yang, Ze Liu, Yifan Xiong, Ziyue Yang, Bolin Ni, Jingcheng Hu, et al. 2023. Fp8-lm: Training fp8 large language models. *arXiv preprint arXiv:2310.18313* (2023).
- [39] Yanghua Peng, Yibo Zhu, Yangrui Chen, Yixin Bao, Bairen Yi, Chang Lan, Chuan Wu, and Chuanxiong Guo. 2019. A generic communication scheduler for distributed DNN training acceleration. In *ACM SOSP*.
- [40] Samyam Rajbhandari, Conglong Li, Zhewei Yao, Minjia Zhang, Reza Yazdani Aminabadi, Ammar Ahmad Awan, Jeff Rasley, and Yuxiong He. 2022. DeepSpeed-MoE: Advancing Mixture-of-Experts Inference and Training to Power Next-Generation AI Scale. In *International Conference on Machine Learning (ICML)*.
- [41] Samyam Rajbhandari, Jeff Rasley, Olatunji Ruwase, and Yuxiong He. 2020. Zero: Memory optimizations toward training trillion parameter models. In *International Conference for High Performance Computing, Networking, Storage and Analysis*.
- [42] Samyam Rajbhandari, Olatunji Ruwase, Jeff Rasley, Shaden Smith, and Yuxiong He. 2021. Zero-infinity: Breaking the gpu memory wall for extreme scale deep learning. In *International Conference for High Performance Computing, Networking, Storage and Analysis*.
- [43] Jeff Rasley, Samyam Rajbhandari, Olatunji Ruwase, and Yuxiong He. 2020. Deepspeed: System optimizations enable training deep learning models with over 100 billion parameters. In *ACM SIGKDD*.
- [44] Jie Ren, Samyam Rajbhandari, Reza Yazdani Aminabadi, Olatunji Ruwase, Shuangyan Yang, Minjia Zhang, Dong Li, and Yuxiong He. 2021. Zero-offload: Democratizing billion-scale model training. In *USENIX ATC*.
- [45] Noam Shazeer. 2020. Glue variants improve transformer. *arXiv preprint arXiv:2002.05202* (2020).
- [46] Noam Shazeer, Azalia Mirhoseini, Krzysztof Maziarsz, Andy Davis, Quoc Le, Geoffrey Hinton, and Jeff Dean. 2017. Outrageously large neural networks: The sparsely-gated mixture-of-experts layer. *arXiv preprint arXiv:1701.06538* (2017).
- [47] Liang Shen, Zhihua Wu, WeiBao Gong, Hongxiang Hao, Yangfan Bai, HuaChao Wu, Xinxuan Wu, Jiang Bian, Haoyi Xiong, Dianhai Yu, et al. 2022. Se-moe: A scalable and efficient mixture-of-experts distributed training and inference system. *arXiv preprint arXiv:2205.10034* (2022).
- [48] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. 2019. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053* (2019).
- [49] Hugo Touvron, Louis Martin, Kevin Stone, Peter Albert, Amjad Almahairi, Yasmine Babaei, Nikolay Bashlykov, Soumya Batra, Prajjwal Bhargava, Shruti Bhosale, et al. 2023. Llama 2: Open foundation and fine-tuned chat models. *arXiv preprint arXiv:2307.09288* (2023).
- [50] TransformerEngine 2025. A library for accelerating Transformer models on NVIDIA GPUs, including using 8-bit floating point (FP8) precision on Hopper and Ada GPUs, to provide better performance with lower memory utilization in both training and inference. (2025). <https://github.com/NVIDIA/TransformerEngine>.
- [51] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. *Neural Information Processing Systems* (2017).
- [52] Shibo Wang, Jinliang Wei, Amit Sabne, Andy Davis, Berkin Ilbeyi, Blake Hechtman, Dehao Chen, Karthik Srinivasa Murthy, Marcello Maggioni, Qiao Zhang, et al. 2022. Overlap communication with dependent computation via decomposition in large deep learning models. In *ACM ASPLOS*.
- [53] Shulai Zhang, Ningxin Zheng, Haibin Lin, Ziheng Jiang, Wenlei Bao, Chengquan Jiang, Qi Hou, Weihao Cui, Size Zheng, Li-Wen Chang, et al. 2025. Comet: Fine-grained Computation-communication Overlapping for Mixture-of-Experts. *arXiv preprint arXiv:2502.19811* (2025).
- [54] Zili Zhang, Yinmin Zhong, Yimin Jiang, Hanpeng Hu, Jianjian Sun, Zheng Ge, Yibo Zhu, Daxin Jiang, and Xin Jin. 2025. Disttrain: Addressing model and data heterogeneity with disaggregated training for multimodal large language models. In *ACM SIGCOMM*.
- [55] Yanli Zhao, Andrew Gu, Rohan Varma, Liang Luo, Chien-Chin Huang, Min Xu, Less Wright, Hamid Shojanazeri, Myle Ott, Sam Shleifer, Alban Desmaison, Can Balioglu, Pritam Damania, Bernard Nguyen, Geeta Chauhan, Yuchen Hao, Ajit Mathews, and Shen Li. 2023. PyTorch FSDP: Experiences on Scaling Fully Sharded Data Parallel. *Proceedings of the VLDB Endowment* (2023).
- [56] Size Zheng, Jin Fang, Xuegui Zheng, Qi Hou, Wenlei Bao, Ningxin Zheng, Ziheng Jiang, Dongyang Wang, Jianxi Ye, Haibin Lin, et al. 2025. Tilelink: Generating efficient compute-communication overlapping kernels using tile-centric primitives. *arXiv preprint arXiv:2503.20313* (2025).

A 附录

A.1 用于参数同步的层次化通信

设完整注意力权重大小为 P ，模型并行（TP 或 SP）维度为 n ，数据并行规模为 d 。通常，用于模型并行的 GPU 位于同一节点内，需要节点内通信；而数据并行跨越多个节点，需要节点间通信。考虑一个包含 d 个设备的数据并行组，其中每个设备持有相同的参数分区。

对于 TP attention 中的参数同步，在 LLM 训练中，通信涉及跨 d 个设备、大小为 P/n 的数据，主要分为两步：

- 节点间 reduce-scatter 操作，数据大小为 P/n ，在 d 个设备上执行。
- 节点间 all-gather 操作，数据大小为 P/n ，在 d 个设备上执行。

因此主要产生节点间通信，通信量为 $2P/n(d-1)/d$ 。

使用 SP attention 时，参数同步涉及跨 $n \times d$ 个设备、大小为 P 的完整数据。考虑节点内和节点间网络带宽的差异，该过程可以通过四步层次化通信实现：复制参数先在节点内归约，再跨节点归约，随后分发回每个设备。图 5a 展示了一个 $n=3$ 、 $d=2$ 的层次化通信示例。具体步骤如下。

- 节点内 reduce-scatter 操作，数据大小为 P ，在 n 个设备上执行。

- 节点间 reduce-scatter 操作，数据大小为 P/n ，在 d 个设备上执行。
- 节点间 all-gather 操作，数据大小为 P/n ，在 d 个设备上执行。
- 节点内 all-gather 操作，数据大小为 P ，在 n 个设备上执行。

SP attention 中的节点间通信量仍为 $2P/n(d-1)/d$ ，额外节点内通信量为 $2P(n-1)/n$ 。

此外，由于节点内和节点间通信使用不同资源，这些步骤可以被切分为小 chunk 并流水线化，以便如图 5b 所示高效地相互隐藏。节点间通信延迟与节点内通信延迟之比为

$$\frac{1}{n} \times \frac{\text{intra-node bandwidth}}{\text{inter-node bandwidth}} \times \frac{n(d-1)}{d(n-1)} \quad (10)$$

考虑一个典型训练场景：使用 H100 SXM 机器，NVLink 带宽为 450 GB/s，设备间 NIC 通信带宽为 50 GB/s。在该背景下，节点间通信延迟很容易超过节点内通信延迟。这意味着节点内通信可以被节点间通信掩盖。因此，在这类场景中，使用 SP attention 同步梯度和参数的表现事实上与 TP attention 一致。